Experiment 1: Breadth First Search (BFS)

Aim:

To implement Breadth First Search algorithm to find the shortest path in an unweighted graph by exploring nodes level by level.

Algorithm:

1. Create a queue and enqueue the start node with path [start].
2. Mark start as visited.
3. While queue is not empty:
   a. Dequeue a node and its path.
   b. If node is goal, return the path.
   c. For each unvisited neighbor, mark visited and enqueue with updated path.
4. If queue empties, no path exists.

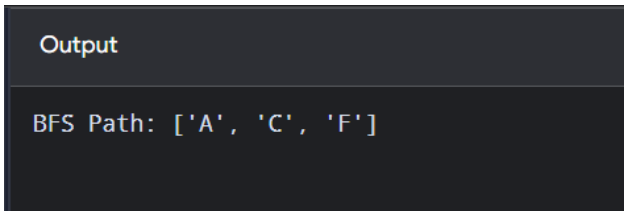
Code:

```
from collections import deque

def bfs(graph, start, goal):
    q = deque([(start, [start])])
    visited = {start}
    while q:
        node, path = q.popleft()
        if node == goal:
            return path
        for n in graph.get(node, []):
            if n not in visited:
                visited.add(n)
                q.append((n, path + [n]))
    return None

graph = {'A':['B','C'], 'B':['D','E'], 'C':['F'], 'D':[], 'E':['F'],
'F':[]}
print("BFS Path:", bfs(graph, 'A', 'F'))
```

Sample Output:



Result:

The BFS algorithm successfully found the shortest path from A to F by exploring level by level. The path ['A', 'C', 'F'] has the minimum number of edges.